ProtocolServer

User Manual

rev. 23/11/2022

Style and Innovation with ClassX

ClassX srl
Via Del Tiglio 91
56012 Calcinaia (PI) ITALY
Tel. Fax. +39 0587 48 81 28
E-Mail: support@classx.it
Home-Page: www.classxcg.com
ORIGINAL EDITION: August 2020

Copyright:
No part of this manual may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording, for any purpose without the express written permission of a duly authorized representative of ClassX srl.

The information contained within this manual is believed to be true and correct at the time of publication.

ClassX, CastaliaCG, MoreCG, CoralCG, LiveBoard, LBScheduler, SGL, TemplateActions, LiveStadium, FrameMarker, PageRecall, SocialServer, SportsGraphics, ThemeChannel are registered trademarks of ClassX srl.

All other brand names are trademarks of their respective holders.

© Copyright 2000-2020 ClassX srl. All rights reserved. Unpublished rights reserved under International copyright laws.

1 Introduction

ProtocolServer has been conceived and designed for external applications which cannot send TCP / IP commands, but which need to interact with ClassX software.
ProtocolServer is a middleware that works between external applications and one of the ClassX graphic playout software such as LiveBoard, LiveBoard LE, StealthCG, LiveBoard SCG or CGTemplateEditor.

To make a real and tangible example of using ProtocolServer, it is sufficient to look at the ClassX MOS solution. In this case ProtocolServer is used to act as the interlocutor between the **ClassX MOSHTML5Plugin** and **ClassX MOSServer**.

2 Installation

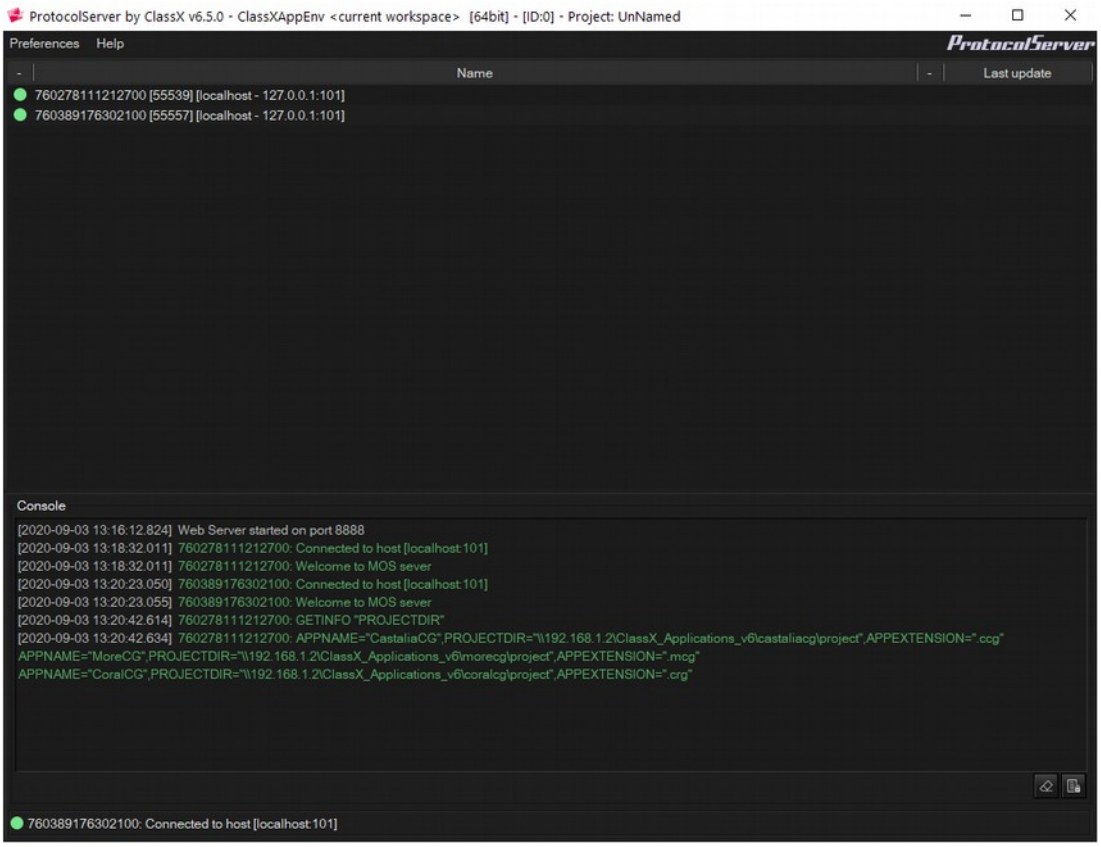
- Download ***ClassX OEM Install 6.x*** from the ClassX dashboard
- Install ***ClassX OEM Install 6.x.exe***
- The software will be installed in C:\ClassX-Applications_v6.x\protocolserver

2.1 Requirements

- It does not need any special requirements to work

3 User interface

The user interface of **ProtocolServer** is very simple.



The user interface is divided horizontally in two parts. The upper part displays the external applications that are connected to the web server of the ProtocolServer, while the lower part is a console used to display the interchange messages between the external applications and the ClassX graphic playout. The bottom "led" icon will be green if there is a connection with the ClassX graphics playout software or red if the connection failed.

3.1 Usage

The **ProtocolServer** is basically a web server capable of handling http requests from several external applications. The web server supports the "**POST**" and "**GET**" methods and supports specific commands in order to use the software in conjunction with other ClassX automation solutions.

To communicate with ProtocolServer it is necessary to know that both requests and responses received and sent by ProtocolServer to and from external applications, take place via the Json format. The first thing that an external application has to do in order to start exchanging information is to **open a session**. For all requests subsequent to opening a session, the external application must always specify the session number. ProtocolServer will provide the required information only for the specified session.

Upon receiving the request from the external application, ProtocolServer transforms it into a TCP/IP command to be sent over the connection opened with the ClassX graphics playout software. ClassX graphics playout software responds to the TCP/IP command over the same connection with the requested data. At this point the ProtocolServer transforms the response of the ClassX graphics playout into Json format and sends it to the external application.

3.2 Additional commands

Being a middleware between external applications and ClassX graphic playouts, ProtocolServer adds some essential features so that external applications can get the best from ClassX protocols.
In fact, there are specific commands provided by ProtocolServer that are added to the normal protocol commands that the ClassX graphic playout software already have.
These commands allow to obtain additional data such as the list of files in a folder, or an image of a graphic template or creating an icon of a graphic template by saving it into a specific folder.
All ProtocolServer-specific commands are prefixed with "**PS_**" except the "**HELP**" command.

The additional commands are the following:

PS_OpenSession

Opens a new session. It returns a new session number to be used for subsequent commands or an error if the command was not successful.

Example:

```
-> 192.168.1.2:8888/{"command": "PS_Opensession"}
{
        "result": [
          "434597459362000"
        ]
}
```

PS_CloseSession <sessionid>

Closes a previously opened connection. An "Ok" message returns or an error if the command was not successful.

Example:

```
-> 192.168.1.2:8888/{"command": "PS_Closesession", "sessionid": "378593255019200"}
{
        "result": [
          "Ok"
        ]
}
```

PS_ExplorePath PATH=path RECURSEDIRS=true/false DIRSONLY=true/false <sessionid>

Returns the list of all files under the PATH folder or an error if the command was not successful. It returns the list of files recursively if RECURSEDIRS is true and only folders if DIRSONLY is true. The arguments RECURSEDIRS and DIRSONLY are not mandatory.

Example:

```
-> 192.168.1.2:8888/{"command": "PS_ExplorePath","args":
["//192.168.1.2/ClassX_Applications_v6/castaliacg/project/"], "sessionid": "1021812136267700

"}
{
        "result": [
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/batsman_caree
r.ccg",
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/batsman_ranki
```

```
ng.ccg",
                "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/batsman_summa
ry.ccg",
                "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/bigfun/bigfun
.ccg",
                  .
                  .
                  .
  ]
}
```

```
-> 192.168.1.2:8888/{"command": "PS_ExplorePath","args":
["//192.168.1.2/ClassX_Applications_v6/castaliacg/project/", "DIRSONLY=true"], "sessionid":
"434729635287200"}
{
        "result": [
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/bigfun",
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/blink",
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/calcio",
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/calcio/livest
adium",
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/Ch1",
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/Ch1-98",
          "//192.168.1.2/ClassX_Applications_v6/castaliacg/project/class_cnbc",
            .
            .
            .
  ]
}
```


PS_GetPublicResources PATH=path RECURSEDIRS=true/false DIRSONLY=true/false <sessionid>

Returns the list of all files under the "public" folder of the web server or an error if the command was not successful. It returns the list of files recursively if RECURSEDIRS is true and only folders if DIRSONLY is true. The arguments RECURSEDIRS and DIRSONLY are not mandatory.

Example:


```
-> 192.168.1.2:8888/{"command": "PS_GetPublicResources","args": ["proxy"], "sessionid":
"434729635287200"}
{
        "result": [
          "C:/ClassX_Applications_v6/protocolserver/home/public/proxy/2D323736363
935353630373234323732303932.png",
          "C:/ClassX_Applications_v6/protocolserver/home/public/proxy/2D343537343
639343933313530343935383438.png",
          "C:/ClassX_Applications_v6/protocolserver/home/public/proxy/2D353935393
733343732373032363432303831 34.png",
          "C:/ClassX_Applications_v6/protocolserver/home/public/proxy/2D37383833
373431313437313632383930313736.png",
          "C:/ClassX_Applications_v6/protocolserver/home/public/proxy/33383631313
532303436333538313236333433.png",
          "C:/ClassX_Applications_v6/protocolserver/home/public/proxy/3939373535
363531343238363534383239 30.png"
        ]
}
```

PS_GetTemplateIcon PATH=path <sessionid>

Returns the image of the template encoded in base64 or an error if the command was not successful.

Example:

```
-> 192.168.1.2:8888/{"command": "PS_GetTemplateIcon","args":
["//192.168.1.2/ClassX_Applications_v6/castaliacg/project/news/lowerthird_news.ccg"],
"sessionid": "434729635287200"}
{
        "result": [
          "iVBORw0KGgoAAAANSUhEUgAAAfQAAAGQCAYAAABYs5LGAAAFPElEQVR42u3WQQ0AQAgDQ
fzLqUEuiLgHZCbpAwVLNQCwVpKuMUcBACsJOgAIOgAg6ACAoAMAgg4Agg4ACDoAIOgAgKADgKADAIIOAAg6ACDo
ACDoAICgAwCCDgAIOgAIOgAg6ACAoAMAgg4Agg4ACDoAIOgAgKADgKADAIIOAAg6ACDoACDoAICgAwCCDgAIOgA
IOgAg6ACAoAMAgg4Agi7oACDoAICgAwCCDgCCLugAIOgAgKADAIIOAAg6AAg6ACDoAICgAwCCDgCCDgAIOgAg6A
CAoAOAoAMAgg4ACDoAIOgAIOgAgKADAIIOAAg6AAg6ACDoAICgAwCCDgCCDgAIOgAg6ACAoAOAoAMAgg4ACDoAI
OgAIOgAgKADAIIOAAg6AAi6oAOAoAMAgg4ACDoACLqgA4CgAwCCDgAIOgAg6AAg6ACAoAMAgg4ACDoACDoAIOgA
gKADAIIOAIIOAAg6ACDoAICgA4CgAwCCDgAIOgAg6AAg6ACAoAMAgg4ACDoACDoAIOgAgKADAIIOAIIOAAg6ACD
oAICgA4CgAwCCDgAIOgAg6AAg6ACAoAMAgg4ACDoAC..."
        ]
}
```

PS_GenerateTemplateProxyIcon PATH=path <sessionid>

Get the template icon, saves it as a png in the "proxy" folder of the web server by making the fnv64 of the template path and it returns the URL of the saved icon or an error in case of problems (the file already exists or does not exist the rights).

Example:

```
-> 192.168.1.2:8888/{"command": "PS_GenerateTemplateProxyIcon","args":
["\\\\192.168.1.2\\ClassX_Applications_v6\\castaliacg\\project\\news\\lowerthird_news.ccg"],
"sessionid": "434729635287200"}
{
        "result": [
          "public/proxy/393835383438313933333838343434393536.png"
        ]
}
```

StoreFile FILENAME=filename BASE64CONTENT=content <sessionid>

Store the supplied file (coded as Base64 string) into shared storage folder on the client machine. This is generally equivalent to {shared.dir}/Stored_Files/<filename_code.ext>.

This call returns the current storage position or error in case of problems (generic i/o errors, bad encoding, etc).

Example:

```
-> 192.168.1.2:8888/{"command":"STOREFILE","type":"REQUEST","params":
[{"name":"FILEPATH","value":"db.PNG"},
{"name":"BASE64CONTENT","value":"iVBORw0KGgoAAAANSUhEUgAABIgAAAN0CAYAAADf2xMvAAAAAXNSR0IArs4c
6QAAAARnQU1BAACxjwv8YQUAAAAJcEh...."}],"sessionid":"1023015048092600"}
{
        "result": [
          "{shared.dir}/Stored_Files/background_AFF5D4E33.jpg>"
        ]
}
```

3.3 Sending commands to ClassX graphic playout

ProtocolServer doesn't know the TCP / IP commands of the ClassX graphic playout, but only reads the command sent by the external application and sends it in TCP / IP to the ClassX graphic playout. In order to do this, the external application must compose the request to the ProtocolServer **web server** following a simple syntax. Here are some examples.

The original command has the following syntax:

GETINFO PROJECTDIR

The command belongs to the protocol used by CGTemplateEditor and has the function of returning the basic paths of the graphic templates.

The command returns the following data:

>APPNAME="CastaliaCG",PROJECTDIR="\\192.168.1.2\ClassX_Applications_v6\castaliacg\project",APPEXTENSION=".ccg"
>APPNAME="MoreCG",PROJECTDIR="\\192.168.1.2\ClassX_Applications_v6\morecg\project",APPEXTENSION=".mcg"
>APPNAME="CoralCG",PROJECTDIR="\\192.168.1.2\ClassX_Applications_v6\coralcg\project",APPEXTENSION=".crg"

As for additional commands, the specific command must be sent to the ProtocolServer using the Json format, so the command changes to:

```
->192.168.1.2:8888/{"command":"GETINFO","type":"REQUEST","params":
[{"name":"INFO","value":"PROJECTDIR"}],"sessionid":"1022708964815200"}
```

The response sent by ProtocolServer is in Json format:

```
{
        "result": [
          "APPNAME=\"CastaliaCG\",PROJECTDIR=\"\\\\192.168.1.2\\ClassX_Applications_v6\\castaliacg\\project\",APPEXTENSION=\".ccg\"",
          "APPNAME=\"MoreCG\",PROJECTDIR=\"\\\\192.168.1.2\\ClassX_Applications_v6\\morecg\\project\",APPEXTENSION=\".mcg\"",
          "APPNAME=\"CoralCG\",PROJECTDIR=\"\\\\192.168.1.2\\ClassX_Applications_v6\\coralcg\\project\",APPEXTENSION=\".crg\""
        ]
}
```

Second example:

The original command has the following syntax:

ADDPROJECT GFXPROJECTPATH, PROJECTID

The command belongs to the protocol used by CGTemplateEditor and has the function of adding a graphic template to the ClassX graphic playout.

The command returns the following data:

>Ok

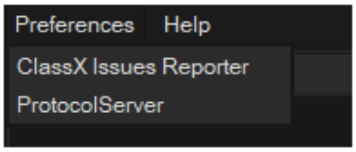
Command transformed into Json:


```
{"command":"ADDPROJECT","type":"REQUEST","params":
[{"name":"GFXPROJECTPATH","value":"{absolute.dir}\\\\192.168.1.2\\ClassX_Applications_v6\
\CastaliaCG\\project\\ibc\\2018\\lowerthird.ccg"},
{"name":"PROJECTID","value":1666012897383}],"sessionid":"1013144611382900"}
```


The answer:


```
{"type":"REPLY","result":["Ok"]}
```


3.4 Preferences



By clicking the "**Preferences**" menu in the upper toolbar, you open the panel with 2 tabs for the management of preferences, through which you can set the startup configuration of the software and make some settings.

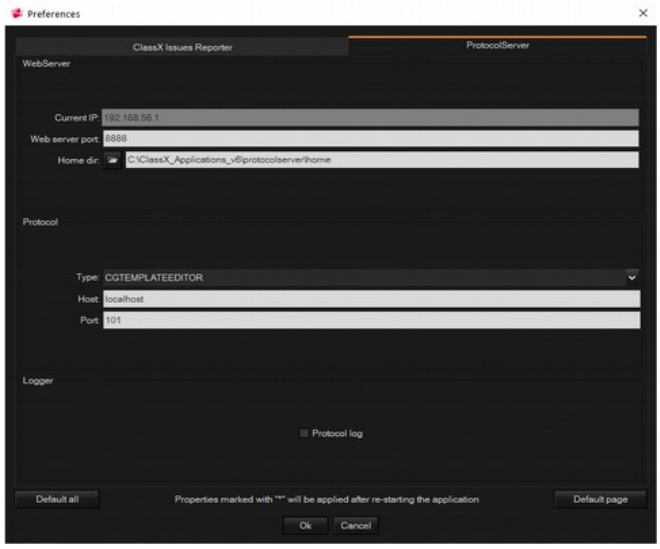
The "**Default Page**" button restores the selected tab to "factory" settings. The button "**Default All**" restores **all tabs** to "factory" settings.

Before exiting from the "Preferences", you have to confirm or cancel the new settings by using the buttons "**OK**" and "**Cancel**".

In ProtocolServer, you can directly access the desired tab of the Preferences Panel by selecting it from the dropdown-menu.

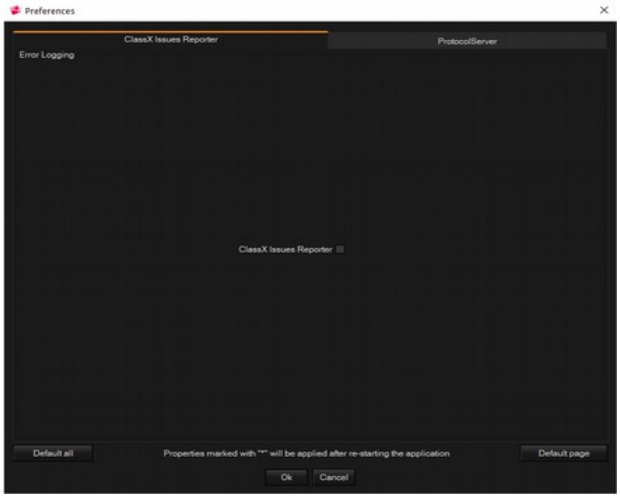
Let's see the single tabs of the **Prefs**:

3.4.1 ProtocolServer



The “**ProtocolServer**” section shows the default properties values of the software used each time the application runs.
The “**WebServer**” section shows the **IP address** of the current workstation and allows to modify the “**web server port**” and his “**home directory**”.
The “**Protocol**” section is dedicated to define the properties for the connection with the ClassX graphic playout.
The “**Type**” chooser allows to select the protocol type to adopt for the connection with the graphic playout.
“**Host**” and “**Port**” fields are used for setting the ID address and port number of the ClassX graphic playout workstation.
The “**Logger**” section shows the unique property for enabling the logging file where all messages between the external applications and the ClassX graphic playout will be visible.

3.4.2 ClassX Issues Reporter:



The “**ClassX Issues Reporter**” tab gives the user the possibility to enable the Issues Reporter, that will send ClassX an error report, should an exception occur.

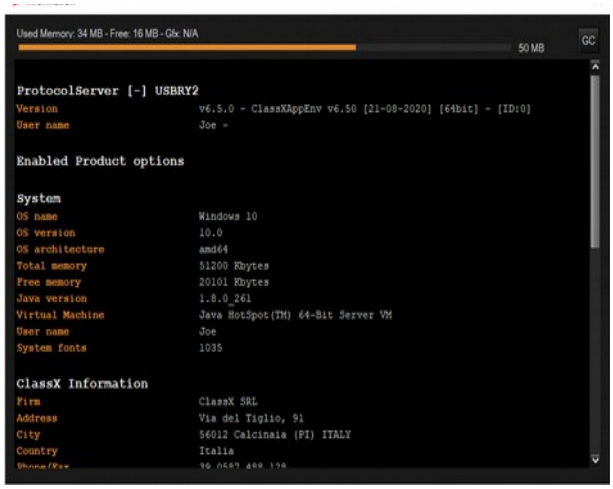
A dialogue will be shown on the Windows tray icons asking the user whether he wants to send ClassX an Error Report or not, if an exception should occur.
No hidden or automatic information will be sent to ClassX in any case, every single Issue sending will have to be approved by the user.

3.5 Help

3.5.1 Information on your installation

The “**Info**” button opens a window that displays some useful infos about the current installation and the state of the program.

3.5.1.1 Memory management



ProtocolServer has been programmed with the precise aim to optimise memory usage. At the program start-up, a certain amount of memory is allocated, enough to satisfy the initial needs of resources loading. ProtocolServer exploits only a part of this memory, using the remaining one (“**free memory**”) in case of need while running the program. If the “free memory” is not enough, the program will allocate more.
The bar above the Information displays the state of the memory allocated by ProtocolServer, showing the used, free and total amount. The orange bar in is the graphic representation of the memory: it represents the used memory, while the dark part represents the free memory. The bar is updated every time you press the “GC” button you may verify the state of memory in real-time at any time.

3.5.1.2 The "GC" button

The term "GC" stands for garbage collection and allows you to perform a particular operation aimed at freeing portions of memory that are no longer in use.

ProtocolServer uses all the resources made available by the operating system in a more optimized way to guarantee maximum performance. However, once the resources are released from ProtocolServer, these are not available immediately but only after a certain period of time for the garbage collector action. The "GC" button starts the garbage collector which allows immediate recovery of the memory. However, this button should be used very carefully because once the garbage collector is in action, it requires a high use of resources and this could lead to drop frames. For this reason it is advisable to use it when there are no contents in execution.

General index

1INTRODUCTION.....3

2INSTALLATION.....3
2.1REQUIREMENTS.....3

3USER INTERFACE.....4
3.1USAGE.....4
3.2ADDITIONAL COMMANDS.....5
PS_OpenSession.....5
PS_CloseSession <sessionid>.....5
PS_ExplorePath PATH=path RECURSEDIRS=true/false DIRSONLY=true/false <sessionid>.....5
PS_GetPublicResources PATH=path RECURSEDIRS=true/false DIRSONLY=true/false <sessionid>.....6
PS_GetTemplateIcon PATH=path <sessionid>.....7
PS_GenerateTemplateProxyIcon PATH=path <sessionid>.....7
PS_StoreFile FILENAME=filename BASE64CONTENT=content <sessionid>.....7
3.3SENDING COMMANDS TO CLASSX GRAPHIC PLAYOUT.....8
3.4PREFERENCES.....9
3.4.1ProtocolServer.....9
3.4.2ClassX Issues Reporter:.....10
3.5HELP.....10
3.5.1Information on your installation.....10
3.5.1.1Memory management.....10
3.5.1.2The "GC" button.....10